

Assignment 3

Learning Outcomes

In this assignment, you will design and implement some new game functionality. This assignment is intended to develop and assess the following unit learning outcomes:

- ✓ **LO1.** Iteratively apply object-oriented design principles to design small to medium-size software systems, using standard software engineering notations, namely UML class diagrams and UML interaction diagrams.
- ✓ **LO2.** Describe the quality of object-oriented software designs, both in terms of meeting user requirements and the effective application of object-oriented design concepts and principles.
- ✓ **LO3.** Apply object-oriented programming constructs, such as abstraction, information hiding, inheritance, and polymorphism, to implement object-oriented designs using a programming language (namely, Java).
- ✓ **LO4.** Apply effective programming strategies to refactor and debug object-oriented implementations systematically and efficiently using available programming language tools.
- ✓ **LO5.** Apply principles of software engineering practice to create object-oriented systems with peers using tools including integrated development environments (IDEs), UML drawing tools, and version control systems.

To demonstrate your ability, you will be expected to:

- design and implement further extensions to the system
- use an integrated development environment to do so
- update your UML class diagrams as required to ensure that they match your implementation
- use git to manage your team's files and documents

The marking scheme for this assignment reflects these expectations.

Introduction



Heavy penalty will apply for editing (add, modify, delete) any classes in the game engine as it breaks the purpose of the whole learning experience.



IMPORTANT: You may change your design if you find that you need to. It is normal for the design to evolve alongside the codebase as the developers' understanding of the requirements improves. If your design changes, you should document the changes. This includes your design rationale as well as your diagrams.

REQ1: The Overgrown Sanctuary

After successfully defeating Abxervyer, The Forest Watcher in the Ancient Woods, the player notices that a new gate is now visible. Unlocking the gate with the old key, the player finds themselves yet in another area, The Overgrown Sanctuary.

```
"++++.....++++.....++++~~~~.....~~~~.....",
"++++.....++.....++++~~~~.....~.....",
"+++.....+++++~.....+++.....",
".....+++++.....+++++.....",
".....++++.....+++++~.....",
".....++.....++.....~.....",
".....+++.....++.....~.....",
"~~~~.....~~~~.....~~~~.....",
"~~~~.....++.....~~~~~.....",
"~~~~.....+++.....~~~~~.....",
"++~.....+++.....+~.....~.....",
"+++.....+++.....+++.....~.....",
"+++.....+++.....+++.....~.....",
"~~~~.....+++.....+++.....~.....",
"~~~~.....+++.....+++.....~....."
```

A sample Overgrown Sanctuary map



The Overgrown Sanctuary must be implemented as a new map (separate from the maps that we have created thus far). The player still needs to unlock a gate and go through it to be able to move to the new map.



Make sure to add another Gate in The Overgrown Sanctuary map to go back to the Abxervyer's map. You can assume that this is a separate gate that the player also needs to unlock.

If you remember, in the Assignment 2 specification, we mentioned that once the boss is defeated, the location where the boss last stood turns into a gate, which leads the player back to the Ancient Woods. Now, refactor the implementation so that this gate leads to two destinations: back to the Ancient Woods and The Overgrown Sanctuary. You may assume that future gates may also have multiple destinations.



Since the implementation of the gate needs to be modified, you need to include your design in the UML class diagram for this requirement.

In the new area, the player notices familiar structures, including graveyards, huts and bushes.

From the graveyard, an old foe, the Hollow Soldier, makes another appearance and starts attacking the player.



You can re-use the implementation of Hollow Soldier from Assignment 1 & 2 (relating to runes) with the exact same specifications, such as they cannot follow the player, they have 200 hit points, and so on.

A new humanoid creature emerges from the huts, which is yet another enemy that the player has to fight against, known as the “Eldentree Guardian” (represented by “e”). The huts can spawn this enemy with a 20% chance at each turn. They have 250 hitpoints and can attack the player with their limbs, dealing 50 damage with 80% accuracy. Once defeated, they have a 25% chance of dropping a healing vial and a 15% chance of dropping a refreshing flask. They also drop 250 runes when defeated. Additionally, they can wander around if the player is not nearby or follow and attack the player if the player is nearby. Finally, they can walk around in the Void with no consequences.

From the bushes, a “Living Branch” (represented by “?”) appears. The bushes can spawn this enemy with a 90% chance at each turn. They have 75 hitpoints and deal 250 damage with 90% accuracy. Once defeated, they have a 50% chance of dropping a Bloodberry. They also drop 500 runes when defeated. They cannot wander around and follow the player. However, they can attack the player if the player is nearby. If they are spawned on top of the Void, they cannot be damaged *by the Void since they are immune*.

Similar to previous Assignments, the new enemies cannot be spawned on or enter floors and walls.

✓ You may want to consider the following points in the design rationale:

- How many classes would you need to create for this requirement if you used the Assignment 1 sample solution?
- Do they contain some code duplications?
- Can you think of a potential design alternative? What would be the cons of the alternative design? Which design did you end up using?

REQ2: The Blacksmith



Please note that the following actor will be added from the start of the game, not only after the player enters The Overgrown Sanctuary.

Back in the building in the Abandoned Village, the player hears the rhythmic sound of a hammer pounding against metal. Following the sound, the player discovers a blacksmith (represented by "B") working in the dark. Approaching the blacksmith allows the player to upgrade their items and weapons.



You can assume that the blacksmith does not move around (that is, they will only stay at one spot in the building).



You can put the blacksmith in the same building where the player first spawns.

When the blacksmith is nearby, the player can upgrade the following items and weapons from their inventory:

- Each healing vial can be upgraded for 250 runes each. When consuming an upgraded healing vial, the player's health will be increased by 80% of their maximum health
- Each refreshing flask can be upgraded for 175 runes each. When consuming an upgraded refreshing flask, the player's stamina will be increased by 100% of their maximum stamina.
- Broadsword can be upgraded for 1000 runes. Each upgrade increases the weapon's damage by 10 points (not increasing the weapon's damage multiplier). To avoid making this weapon too powerful, the additional damage points from the upgrades are not affected by the damage multiplier. That is, only the initial 110 damage points of the broadsword will be multiplied by the damage multiplier, while the damage points obtained from performing the upgrades will simply be added on top of the multiplied damage points. So, if the player uses the focus skill on the broadsword that has been upgraded once, then the final damage points will be 131, which is obtained from $(110 * 1.1) + 10$
- Great Knife can be upgraded for 2000 runes. Each upgrade increases the weapon's hit rate by 1 per cent.
- Giant hammer cannot be upgraded.

The player cannot sell any items to the blacksmith, and the player cannot attack the blacksmith.

Update (9/10/2023):

Healing Vial and Refreshing Flask can only be upgraded once. Once Healing Vial or Refreshing Flask have been upgraded, the player should not get an option to upgrade them.

On the other hand, Broadsword and Great Knife can be upgraded multiple times.

REQ3: Conversation (Episode I)



Please note that this functionality will be active from the start of the game, not only after the player enters The Overgrown Sanctuary.

As the player awaits their weapons and items upgrades to complete, the blacksmith decides to share their story and experience on the land of the lords with the player.

When the blacksmith is nearby, the player should be given the option to listen to the monologue of the blacksmith. What the blacksmith says will be chosen randomly from the following list of monologue options below. Note that some options can only be performed when certain conditions are met.

The blacksmith's monologue options:

- I used to be an adventurer like you, but then Nevermind, let's get back to smithing.
- It's dangerous to go alone. Take my creation with you on your adventure!
- Ah, it's you. Let's get back to make your weapons stronger.
- **If the player hasn't defeated Abxervyer:** Beyond the burial ground, you'll come across the ancient woods ruled by Abxervyer. Use my creation to slay them and proceed further!
- **If the player holds a great knife:** Hey now, that's a weapon from a foreign land that I have not seen for so long. I can upgrade it for you if you wish.
- **If Abxervyer has been defeated:** Somebody once told me that a sacred tree rules the land beyond the ancient woods until this day.

For example, if the player hasn't defeated Abxervyer, hasn't purchased a Great Knife and chooses to talk to the blacksmith, possible monologue options are 1, 2, 3, and 4. Once they buy a great knife before defeating Abxervyer, option 5 will be added to the pool of options. Once they defeat Abxervyer, option 4 will be removed, while option 6 will be added to the list of options.

REQ4: Conversation (Episode II)



This functionality is a continuation of REQ 3

Additionally, the player should be able to listen to the monologue of the isolated traveller found in the Ancient Woods (see *Assignment 2 REQ 3*). Similarly, what the traveller says will be one option from the following list of options, chosen at random. Some monologue options can only be performed when certain conditions are met.

Isolated traveller's monologue options:

- Of course, I will never give you up, valuable customer!
- I promise I will never let you down with the quality of the items that I sell.
- You can always find me here. I'm never gonna run around and desert you, dear customer!
- I'm never gonna make you cry with unfair prices.
- Trust is essential in this business. I promise I'm never gonna say goodbye to a valuable customer like you.
- Don't worry, I'm never gonna tell a lie and hurt you.
- **If the player holds a Giant Hammer:** Ooh, that's a fascinating weapon you got there. I will pay a good price for it. You wouldn't get this price from any other guy.
- **If the player hasn't defeated Abxervyer:** You know the rules of this world, and so do I. Each area is ruled by a lord. Defeat the lord of this area, Abxervyer, and you may proceed to the next area.
- **Once the player defeats Abxervyer & they still hold the giant hammer:** Congratulations on defeating the lord of this area. I noticed you still hold on to that hammer. Why don't you sell it to me? We've known each other for so long. I can tell you probably don't need that weapon any longer.

REQ5: A Dream?



Please note that starting from this requirement, whenever and wherever the player dies, the functionality specified below will occur. In other words, this functionality will be active from the start of the game, not only after the player enters The Overgrown Sanctuary.

During the battle, the player makes a small mistake and gets knocked out unconscious by an enemy.

The player suddenly awakens right back in the building in the Abandoned Village where their journey starts, realising that everything so far was all just a dream - or is it?

The Assignment 1 specification mentions that if the player dies due to any causes, the game ends, and a "YOU DIED" message is displayed. In this requirement, you should refactor the implementation as follows:

When the player dies, they will be respawned right back where they started their journey, which is in the building in the middle of the Abandoned Village (see Assignment 1 specification).

The maximum values of the player's attributes will be kept as what it was before the player dies, while the value of the attributes will be reset to full. For instance, the player spawns with (150/150) hitpoints. Then, they consumed a bloodberry, which increased their maximum health to 155. Once they respawned, their hitpoint should be (155/155) instead of (150/150). This also applies to their other attributes, such as stamina.

The player should still keep every item that they carry in their inventory. For example, if in the previous run of the game, the player picks up the broadsword, healing vials and runes, they get to keep all of them when they respawn.

However, the runes in the player's wallet (not in their inventory) will be reset back to 0. These runes should be dropped in the last location where the player last stood. For example, if the player carries 5 runes items in their inventory and has a 5000 runes balance in their wallet, when they respawn, they get to keep the 5 runes items, but their balance will be reset to 0. In the player's last location, a new runes item will be dropped containing the 5000 runes balance.

For simplicity, even if the runes balance in the player's wallet is 0, you can still drop it on the ground with 0 amount.



Don't worry if the player dies due to stepping into the Void. The runes can still be dropped in the location where the player last stood (the void). It is simply not recoverable. Optionally, you could drop the runes at the location before the player enters the void. However, since this only improves the gameplay, it will not be considered when marking.

All spawned enemies (not including bosses) will be removed from the map.

For bosses, their health will be reset to full if they have not been defeated.

Unlocked gates will be locked again.

All runes dropped in previous turns that the player did not pick up will also be wiped from the map. This includes all runes dropped by enemies and the player. For instance, in the previous turn, the player dies while carrying X amount of runes, the player's wallet balance will be reset to 0, and the X amount of runes are dropped in the last location where the player stood. However, before picking up the X amount of runes previously dropped, the player dies again. Then, the X amount of runes will be wiped from the game, as they were not picked up by the player.

Submission Instructions



Within this class, you are welcome to use foundation models (ChatGPT, GPT, DALL-E, Stable Diffusion, Midjourney, GitHub Copilot, and anything after) in a totally unrestricted fashion, for any purpose, at no penalty. However, you should note that all large language models still have a tendency to make up incorrect facts and fake citations, code generation models have a tendency to produce inaccurate outputs, and image generation models can occasionally come up with highly offensive products. You will be responsible for any inaccurate, biased, offensive, or otherwise unethical content you submit regardless of whether it originally comes from you or a foundation model. If you use a foundation model, its contribution must be acknowledged in the submission (in the form of a separate file titled `FoundationModelsUsed.txt`); you will be penalised for using a foundation model without acknowledgement.

Having said all these disclaimers, the university's policy on plagiarism still applies to any uncited or improperly cited use of work by other human beings, or submission of work by other human beings as your own. Moreover, missing contribution logs/GIT commits and/or failing any handover interview will be treated as a potential breach of academic integrity which will be further investigated.



We will mark your assignment based on the latest commit in the `main` branch in your GitLab repository by the due date, **not** in the `master` branch.



Heavy penalty will apply for editing (add, modify, delete) any classes in the game engine as it breaks the purpose of the whole learning.

You are expected to produce the following artefacts in Assignment 3:

1. Revised (based on feedback received on Assignment 2 or if you consider changes were needed)
+ New UML design documents, which include
 - 5 UML class diagrams (1 for each requirement, similar to Assignments 1 and 2)
 - 5 Interaction diagrams (1 for each requirement: sequence diagrams or communication diagrams are equally acceptable)
2. Design rationale, which should include (5 of them, 1 for each requirement)
 - Justification for any changes made to the design proposed for Assignment 2
 - Justification for the new requirements
3. Implementation (the code) of the system.
4. Contribution Log (mandatory, useful for markers) - see <https://edstem.org/au/courses/12156/lessons/37194/slides/259339>



For class diagrams, please make sure to distinguish between existing classes and new classes with different colours. Additionally, you should colour revised classes and newly added classes differently.

Additionally, you will be asked to complete the following forms:

- Peer Assessment Form (mandatory, useful for markers and for you to reflect on your group dynamics)

- Feedback Request Form (to be completed in order to upload your files to Moodle).

As mentioned above, you can (and should) update the artefacts previously marked in Assignment 2 since the code must align with the UML diagrams.

We will mark your Assignment 3 on the state of the `main` branch of your Monash GitLab repository by the due date at your local time (see above). If you have done any work in other branches, make sure you merge it into `main` before the due date. Additionally, please, make sure at least one team member submits the assignment to Moodle. This step is compulsory.



Do not create a new copy of your work for Assignment 3. Continue working on the same files, in the same directory structure (you might like to add a tag to your final Assignment 2 commit before starting on Assignment 3, so you can find that version easily). <https://git-scm.com/book/en/v2/Git-Basics-Tagging>

As we said above, you may update your design and implementation if you find that your Assignment 2 solution is not suitable for extension or if you think of a better approach during Assignment 3.

It is highly recommended that you explicitly Indicate in the **design rationale** what changed in the design between Assignment 2 and Assignment 3. Explaining the changes would help you organise your ideas to explain how the design changed during your handover video recording. A quick summary of changes with some explanation regarding why you applied such changes would be really valuable.

Contribution Log

We require you to have a document to know how you plan to divide the work between team members and simultaneously capture individual contributions. We have prepared a template you can copy to your university's Google Drive and share with your team. Please follow the detailed instructions [here](#).



Since your markers need to access this information, make sure that you create a share link and put it in the README of your repository. Failure to do so will result in 0 mark.

In this matter, everyone must contribute a **FAIR amount of code AND documentation**. It means you cannot work only on the code or only on documentation. In other words, your team **MUST NOT** split the work by artefacts: a person working on a "class diagram", another on a diagram alone, and another working on design rationale alone. We will give a heavy penalty to your team as a whole.

You will meet up with your team ideally once a week for about 1 hour for team discussion and another 1 or 2 hours for integration and resolving conflicts (if any).

Peer Assessment Form

You should also complete the peer assessment form, which you can find here:

<https://forms.gle/G3J2sMqk8G5X5aEb8>, to help us understand and evaluate your (and other team members') contributions to the assignment better.

About Late Submissions

Unless a team member has applied for and received special consideration according to the [Monash Special Consideration Policy](#), late submissions will be penalised at 10% per day late (including weekends). Special considerations are granted individually.

It is all team members' responsibility to ensure that the correct versions of the documentation and code are present in the repository by the due date and time.

Handover Video Instructions

For Assignment 3 (A3), we will NOT be conducting a handover interview. Instead, we kindly request that you INDIVIDUALLY record a short video where you succinctly describe your work, placing particular emphasis on the good design practices you applied in the last assignment.

Please adhere to the instructions outlined below:

1. Upload your video presentation as an unlisted video on YouTube. Alternatively, the video could also be uploaded to your student account's Google Drive, Panopto or Zoom Cloud.
2. Your presentation **should not exceed 4 minutes in duration per student**. So, if you are a group of 4, the presentation should not exceed 16 minutes in length, while If you are a group of 3, it should not exceed 12 minutes.
3. The primary aim is for you to INDIVIDUALLY demonstrate **how you contributed to any of the requirements of A3**. Additionally, you should explain the correctness of your solution **based on the design principles** discussed in the unit.



Just a friendly reminder that when submitting the handover video, please do not only run the game but also explain your code and demonstrate a good understanding of your implementation. If the video only shows the game without any explanation or understanding, the entire group will be flagged as an academic integrity case that may require further investigation.

4. During the entirety of your video presentation/recording, ensure that your webcam is enabled and **your face is clearly visible** without the use of background images or filters.



If the webcam is turned off or the face is not clearly visible in the video recording, we will also flag this as an academic integrity case since we cannot verify if it's you who explained the solution. This may require further investigation.

5. You must verbally discuss your solution and the rationale behind it while **screen-sharing your code in IntelliJ**.
6. There's no need for professional video recording and editing tools; **a simple recording via Zoom with screen share will suffice**.
7. Kindly create a text file and upload it to GIT (and Moodle) titled "**HandoverVideos.txt**". This file should contain the links to the videos of each member of your team in the following format:

Student Name 1 - Link to Zoom/Google Drive/Panopto/unlisted YouTube video

Student Name 2 - Link to Zoom/Google Drive/Panopto/unlisted YouTube video

Student Name 3 - Link to Zoom/Google Drive/Panopto/unlisted YouTube video

... and so on.

After you've received your final scores for this unit, you may remove the video from the platform where you uploaded it.



Please make sure that the teaching team can access the video on the platform where you uploaded it to. For instance, if you upload the video on Google Drive, please make sure to give the correct permission so that the teaching team can correctly access the video.

Please note that the teaching team is not expected to contact you if the videos are not accessible.

Marking Rubric

Assignment 3 Rubric

Prerequisite for marking the whole assignment: Design documents (UML diagrams and design rationale) and implementation need to be submitted for the assignment to go through the marking process. Missing any of these will result in 0 marks.

Overview:

In this assignment, the basic requirements (REQs 1-4) and the challenging requirement will be marked separately (REQ 5).

Basic requirements are worth 80% of the marks (REQs1-4)

26 points

- Feature implementation completeness (6 points)
- Implementation quality: design principles (8 points)
- Design rationale (4 marks)
- Integration with the existing system (2 points)
- UML syntax and clarity (2 points)
- Alignment and design consistency (2 points)
- Style & Javadoc (1 point)
- Format and directory structure (1 point)

The challenging requirement is worth 20% of the marks (REQ 5)

24 points

- Feature implementation completeness (6 points)
- Implementation quality: design principles (8 points)
- Design rationale (4 points)
- Integration with the existing system (2 points)
- UML syntax and clarity (2 points)
- Alignment and design consistency (2 points)

Detailed rubric:

Feature implementation completeness (6 points)

6 marks - The system runs and perfectly meets **all functional expectations** as per the relevant requirement(s) with no runtime errors. All required classes and relationships are implemented, and the program's behaviour aligns perfectly with the specification.

5 marks - The system runs and meets **all functional expectations (with some minor errors or punctual omissions)** as per the relevant requirement(s). All necessary classes and relationships are included, and the program's behaviour largely matches the specification, except for **one or two minor unexpected behaviours**. No runtime errors occur.

4 marks - The system runs and **partially meets all functional expectations, displaying several minor functional errors or omissions** as per the relevant requirement(s). Despite this, the majority of necessary classes and relationships are included, and **only a few minor unexpected behaviours occur**. No runtime errors are present.

3 marks - The system runs and **all functional expectations were addressed to some extent**, (with one or two major functional errors) as per the relevant requirement(s). Most important classes and relationships are included but **at least one or two major unexpected behaviours** are observed. A major unexpected behaviour is that which affects considerably the completeness of at least one requirement. No runtime errors occur.

2 marks - The system runs but **several functional expectations were not addressed** as per the relevant requirement(s) (e.g. the notion of Behaviour is not included even though it is a part of the requirement). Alternatively, runtime errors occur during the execution of the system but they can be easily fixed.

1 mark - The system runs but **addresses only some functional expectations and shows major omissions** as per the relevant requirement(s) (important classes or relationships are missing). Alternatively, runtime errors occur during the execution of the system without a clear idea of the cause or cannot be easily fixed.

0 marks - The system runs but it **poorly addresses or doesn't address the functional expectations** as per the relevant requirement(s). Alternatively, the system might not run at all.

Implementation quality: design principles (8 points)

This item applies across all functional expectations as per the relevant requirement(s)

8 marks - The implementation of all functional expectations as per the relevant requirement(s) **flawlessly adheres to good design principles and concepts** (e.g., DRY, SOLID principles and connascence), making the design easy to extend and maintain. Any punctual violations are **convincingly** justified in the design rationale (e.g., using singleton to implement a feature). All relevant requirements have been addressed.

7 marks - The implementation follows good design principles **nearly perfectly** making the design is easy to extend and maintain (if some punctual principles are violated, the trade-off is **somewhat** justified in the design rationale. All relevant requirements have been addressed.

6 marks - The implementation involves **one or two minor violations of design principles** (could be easily fixed) across all functional expectations as per the relevant requirement(s) (e.g., some attributes are not set to private without any justification). All relevant requirements have been addressed.

5 marks - The implementation involves **minor violations of design principles in multiple places** (could still be easily fixed) across all functional expectations as per the relevant requirement(s). All relevant requirements have been addressed.

4 marks - The implementation involves **one or two non-severe violations of design principles that could be implemented in a better way** (no trade-offs are convincingly provided in the design rationale) across all functional expectations as per the relevant requirement(s) (e.g. some code repetitions are found in the implementation that would require some refactoring to be fixed). All relevant requirements have been addressed.

3 marks - The implementation involves **non-severe violations of design principles in multiple places** (no trade-offs are convincingly provided in the design rationale). All relevant requirements have been addressed.

2 marks - The implementation involves **one or two severe violations** of design principles that could be implemented in a better way across all functional expectations as per the relevant requirement(s), e.g. violating the SOLID principles. Fixing them would require substantial refactoring. Alternatively, not all relevant requirements have been attempted.

1 mark - The design/implementation can be considered **hacky or various instances of procedural programming are found**. For example, the implementation uses downcasting and 'instanceof' in various cases without a convincing justification. Alternatively, it can also be the case that abstraction is not used, making the design **difficult to extend and maintain**. Alternatively, only some relevant requirements have been attempted.

0 marks - The implementation mainly follows a non-OO paradigm; or the UML class diagram(s) or the implementation is missing. Alternatively, most relevant requirements have not been attempted.

Design rationale (4 points)

4 marks - The design rationale includes a description of **what has been done and why**, focusing on the principles and concepts taught in the unit (such as DRY, SOLID principles, - **with a particular emphasis on connascence, and code smell**), and not in terms of game design. The rationale discusses the **advantages and disadvantages** of the design (pros and cons), **the reasons** for choosing the current design, and **ways in which it can be easily extended** (e.g. my design achieves OCP because if a new character is added in the future ...). All relevant requirements have been addressed.

3 marks - The rationale describes **what has been done and why**, based on the principles and concepts taught in the unit. It also includes **some discussion** of the pros and cons of the design and the reasons for the current design choice but **lacks examples of future extensibility**. All relevant

requirements have been addressed.

2 marks - The rationale describes **what has been done and why**, aligning with the principles and concepts taught in the unit. However, it **lacks a discussion on the pros and cons of the design and/or examples of future extensibility**. All relevant requirements have been addressed.

1 mark - The rationale **primarily describes what has been done** without a thorough explanation of the decision-making reasons. For instance, it may mention design principles and concepts without providing convincing explanations, discussion of pros and cons, or examples of how the system can be extended. Alternatively, not all relevant requirements have been attempted.

0 marks - The design rationale is either very challenging to read, is missing, or the submitted work omits more than one relevant requirement.

Integration with the existing system (2 points)

This item applies across all relevant requirements.

2 marks - The implementation **effectively uses the engine classes** (e.g. the submitted work demonstrates that the students understand the difference between actions and behaviours). All relevant requirements have been addressed.

1 mark - The implementation **does not use some engine classes as intended** (e.g. behaviours are created for some actions that do not need it, such as actions performed by the player) or includes **custom classes for functionality that could be implemented with the engine class** (e.g. created a custom ground class instead of using the ground class given in the engine package). Most relevant requirements have been addressed.

0 marks - The engine has been modified in a minor or significant way OR the UML class diagram OR the implementation is missing

UML syntax and clarity (2 points)

This item applies across all relevant requirements.

2 marks - Relevant (static and/or dynamic) diagrams are **perfect in terms of syntax**, with no missing multiplicities, correct arrowheads for all relationships, and appropriate usage of realisation for classes implementing interfaces, generalisation for classes or interfaces inheriting others, etc. Diagram formatting is **consistent and clear**. All requested diagrams have been submitted.

1 mark - Diagram(s) contain **some minor syntax errors**, such as missing multiplicities for several associations, inappropriate use of generalisation for classes implementing interfaces, realisation for classes extending others, or classes extending multiple classes, etc. Nonetheless, the design can still be understood by the TA with a little effort. Alternatively, there are several **inconsistencies across diagrams**. All necessary diagrams have been submitted.

0 marks - Diagram(s) are significantly hard to understand or show major inconsistencies in formatting

and clarity, OR more than one required diagram is missing, OR they do not resemble a UML diagram as required.

Alignment and design consistency (2 points) - incl. Design rationale, code and UML diagrams

2 marks - The design rationale and UML diagrams are in **perfect alignment** with each other and with the code implementation across all functional expectations, as per the relevant requirement(s). All relevant requirements have been addressed or attempted.

1 mark - There are **some small inconsistencies** (that can be easily fixed) between the design documents and the implementation, but all relevant requirements have been addressed or attempted.

0 marks - There are **major inconsistencies** (which would necessitate significant changes for correction) or **numerous smaller discrepancies** distributed throughout the design documents and the implementation. Alternatively, more than one required diagram or implemented requirement is missing, or some submitted design documents do not resemble the required UML diagram format.

Style & Javadoc (1 point)

Applicable to basic requirements (1-4) only

1 mark - The code is properly documented with Javadoc across all functional expectations as per the relevant requirement(s), including class-level and method-level documentation. The Google Java Style guide is followed properly (e.g. package names are written in lowercase, attributes and variables names are written in lowerCamelCase, class names are written in UpperCamelCase, etc.). All relevant requirements have been addressed or attempted.

0.5 marks - Some classes and methods are missing Javadoc documentation OR some classes do not follow the Google Java style guide

0 marks - Most classes and methods are missing Javadoc documentation OR The Google Java style guide is not followed.

Project formatting and directory structure (1 point)

Applicable to basic requirements (1-4) only

1 mark - If the directory structure suggested in the specification has been followed. All the documents are easily found where expected.

0 marks - The suggested directory structure was not followed and some files may be hard to find.

Individual Contribution (100%) & Handover Video

(compulsory)

We will, in principle, assume all team members equally contributed to the assignment. Yet, this individual contribution component will be used to weigh the marks awarded at a group level (see above) for cases where individual students did not contribute to the group task fairly (see details below) or did not show evidence that they understand their own work during the handover interview (see details under Handover Interview, below).

If you contributed to the group task and demonstrated that you understand your own work you should get (100%) the full marks awarded at a group level as per the rubric above.

Individual Contribution (100%) Only for A2 and A3

100-80% - The student has made substantial contributions to the submitted work, evident by numerous **commits** (over 10) with meaningful commit comments (note: default comments from web UI are not considered). For full marks, there should be **at least one commit per week leading up to the submission deadline, and multiple contribution log entries** (over 10 entries) with comprehensible explanations for an external reader (e.g. the TA).

Additionally, the student's contributions reflect a reasonable understanding of OOP principles, whether in programming or designing UML diagrams. The student's input spans **both design and code**.

Additionally, other team members have given positive feedback or there are no complaints regarding the student's contributions (as **per peer assessment**).

70-40% - The student has made some contributions to the submitted work, evident by **some commits** (between 5 and 9) with meaningful commit comments OR the student made more than 10 commits but **all of them on the same week of the submission deadline** or there may be **commits without any comments**.

Alternatively, there are **some contribution log entries** (between 5 and 9) with comprehensible explanations for an external reader (e.g. the TA).

Alternatively, the student's input focused **only on the design or the implementation**.

Alternatively, there are **some complaints** regarding the student's contributions (as per peer assessment).

30-20% - Peer assessment, contributions logs, and git logs show there are **insufficient contributions from an individual**; the student received **a negative review** from other team members which usually happened due to last-minute work, difficult communication outside of labs, or any other kind of difficulties (evidence are collective from peer assessment, emails, contribution logs, and git logs). The student committed to git, with **at least four commits** and meaningful comments OR there are various commits that are not meaningful and that do not explain the purpose. **Some contribution log entries** (at least 4 entries) with meaningful explanations that can be understandable by a new team member (e.g. the TA).

10-0%% - There is no or little evidence of contribution to the team from Git logs or contribution logs. If applicable, the student only writes comments (inline or Javadoc) without any appropriate programming contribution. Clear evidence of academic misconduct of an individual would be marked as zero. Note: submitting files as attachments to the release notes would be marked as zero.

INDIVIDUAL Handover Video (compulsory)

COMPLETION If the student can briefly describe what parts of the system they created and briefly describe design decisions in a video no longer than 4 minutes. The short video needs to demonstrate that the student understands the various parts of the assignment (as per the logged commits) and at least some awareness of those parts created by other team members.

NOT COMPLETED If the video is not presented (see specifications) or it is unclear whether the student understands their own work from the video.



IMPORTANT: Failing to have meaningful commits (i.e. showing that the task was progressively completed) and/or failing to provide the handover video would automatically flag this as a potential case of plagiarism, it will be further investigated using a similarity check software, and **zero marks would be awarded.**



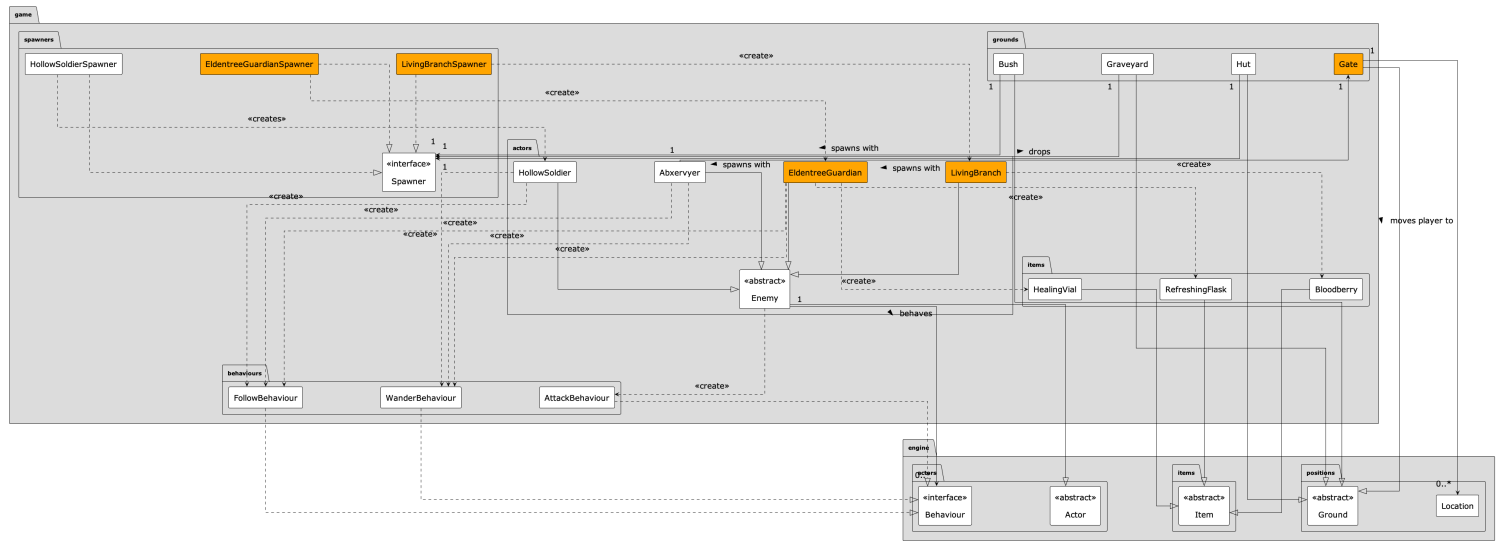
IMPORTANT: Any inquiry (e.g. potential conflict within a team) should be submitted via the emails below (not your AdminTA, Lecturer nor CE). Emails sent in other ways will not be processed in time.

FIT2099.Clayton-x@monash.edu if you are based in Clayton

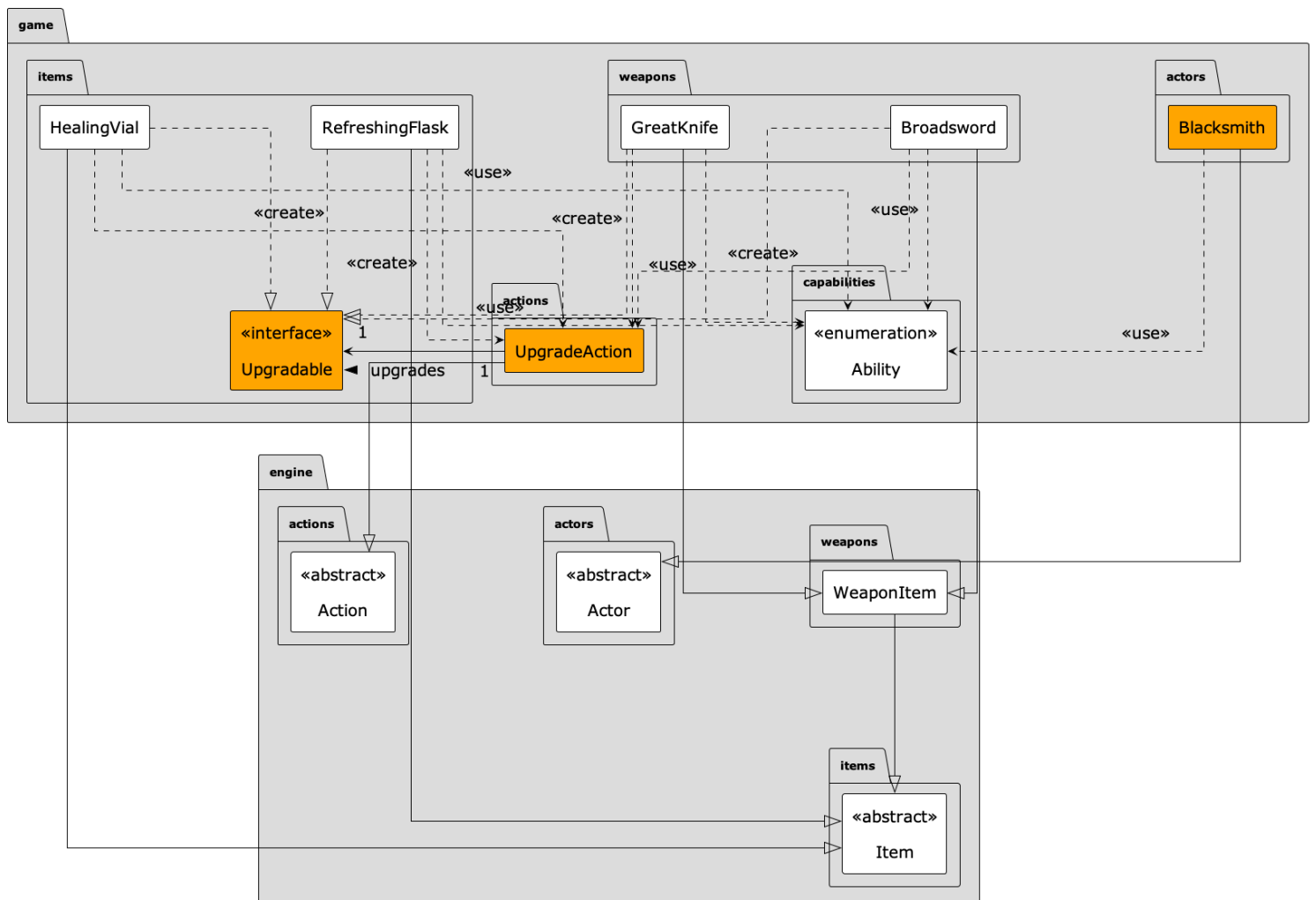
FIT2099.Malaysia-x@monash.edu if you are based in Malaysia

Assignment 3 Sample UML class diagrams

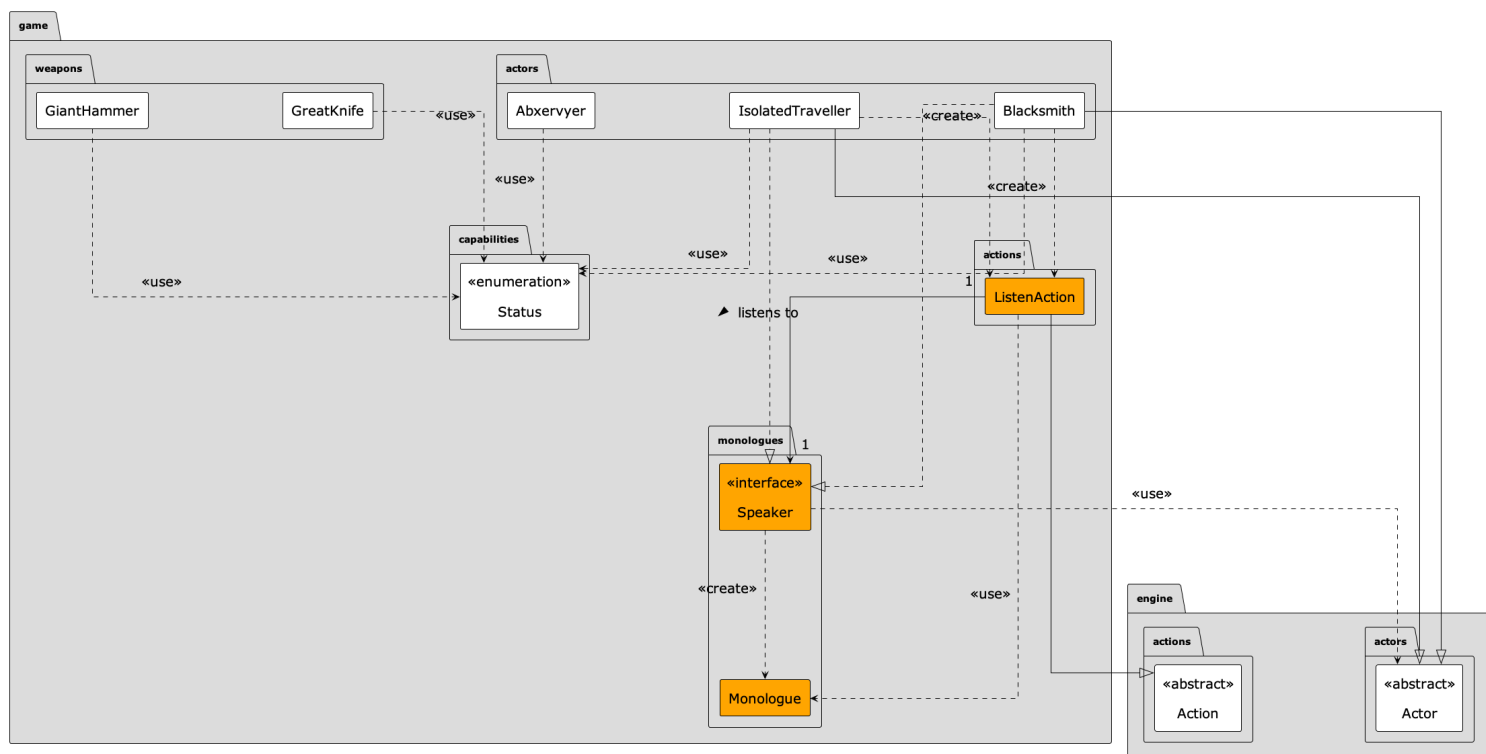
Requirement 1



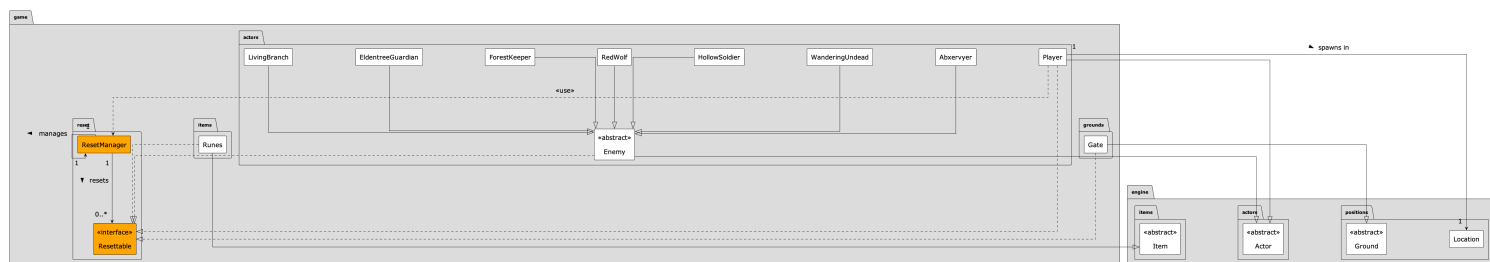
Requirement 2



Requirement 3 & 4



Requirement 5



Assignment 3 Q&A Session

Direct Link: <https://monash.au.panopto.com/Panopto/Pages/Viewer.aspx?id=812f07a4-8e5d-40af-8f89-b09f003d70f4>

Meeting chat:



GMT20231013-041050_RecordingnewChat.txt

